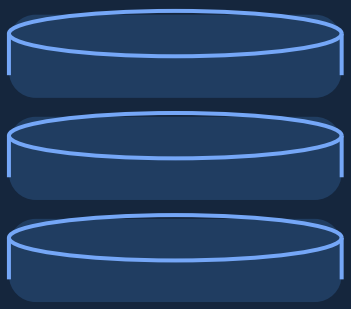

GUIA DE ESTUDO • FUNDAMENTOS DE BANCO DE DADOS

Conceitos, regras, exemplos práticos e boas práticas

Um guia objetivo para compreender SQL, normalização, chaves e relacionamentos.

01 | Linguagem SQL

SQL (Structured Query Language) é a linguagem usada para definir estruturas, consultar dados e realizar alterações em bancos de dados relacionais. Os comandos costumam ser organizados por finalidade.

GRUPO	FINALIDADE	COMANDOS COMUNS
DDL — Data Definition Language	Criar ou alterar a estrutura do banco.	CREATE, ALTER, DROP, TRUNCATE
DML — Data Manipulation Language	Inserir, modificar ou excluir registros.	INSERT, UPDATE, DELETE
DQL — Data Query Language	Consultar e recuperar informações.	SELECT
DCL — Data Control Language	Controlar permissões de acesso.	GRANT, REVOKE
TCL — Transaction Control Language	Controlar transações.	COMMIT, ROLLBACK, SAVEPOINT

Exemplo rápido

```
CREATE TABLE aluno (id INT PRIMARY KEY, nome VARCHAR(100));
```
Define uma tabela chamada **aluno**.

```
INSERT INTO aluno (id, nome) VALUES (1, 'Ana');
```
Insere um registro.

```
SELECT id, nome FROM aluno;
```
Consulta os registros.

Boa prática: antes de executar UPDATE ou DELETE em dados importantes, confirme a condição WHERE e, quando possível, faça a operação dentro de uma transação.

02 | Normalização de dados

Normalizar é organizar tabelas para reduzir redundância e evitar anomalias de inserção, atualização e exclusão. A normalização é feita por etapas; cada forma normal acrescenta critérios à anterior.

Visão geral das formas normais

FORMA	REGRA CENTRAL	SINAL DE ALERTA
1FN	Cada campo contém um valor atômico; não se armazenam listas ou grupos repetidos na mesma célula.	Colunas como telefone1, telefone2, telefone3 ou valores separados por vírgula.
2FN	Está na 1FN e cada atributo não-chave depende da chave primária inteira, sem dependência parcial de uma chave composta.	Em uma tabela cuja chave é (id_pedido, id_produto), o nome do produto depende apenas de id_produto.
3FN	Está na 2FN e não há dependências transitivas de atributos não-chave em relação à chave.	Guardar nome e telefone do departamento repetidos em cada funcionário.
BCNF	Versão mais restritiva: todo determinante não trivial deve ser uma superchave.	Dependências funcionais que ainda geram anomalias mesmo após a 3FN.

Exemplo de 1FN

Em vez de guardar “(31) 99999-1111, (34) 98888-2222” em uma única coluna de aluno, mantenha os telefones em registros separados numa tabela relacionada. Assim, cada valor ocupa seu próprio campo e pode ser consultado individualmente.

Importante: normalização não significa simplesmente “criar mais tabelas”. O objetivo é representar corretamente os fatos e as dependências entre os dados.

03 | Primeira Forma Normal (1FN)

Uma tabela está na 1FN quando cada célula contém um único valor do domínio esperado, não há grupos repetidos e cada registro pode ser identificado de forma única.

Antes: valores múltiplos em uma coluna

id_aluno	nome	telefones
1	Ana	(31) 99999-1111; (34) 98888-2222
2	Bruno	(11) 97777-3333

Depois: dados atômicos em tabelas relacionadas

ALUNO	TELEFONE
id_aluno (PK) nome	id_telefone (PK) id_aluno (FK) numero
1 • Ana	101 • 1 • (31) 99999-1111 102 • 1 • (34) 98888-2222
2 • Bruno	103 • 2 • (11) 97777-3333

Por que melhora?

• Permite adicionar ou remover um telefone sem alterar o registro do aluno.
• Facilita buscas, validações e índices por número.
• Evita que consultas precisem interpretar listas dentro de um campo.

A 1FN também exige uma forma de distinguir os registros. Na prática, costuma-se definir uma chave primária adequada para cada tabela.

04 | Chaves primárias: natural e artificial

A chave primária (PK, Primary Key) identifica cada linha de uma tabela. Ela deve ser única e não pode ser nula. A escolha da chave influencia a integridade e a manutenção do modelo.

TIPO	O QUE É	EXEMPLO	PONTOS DE ATENÇÃO
Chave natural	Atributo do mundo real que já possui significado e pode identificar o registro.	Código oficial estável, ISBN, código de produto padronizado.	Pode mudar, ter regras externas ou exigir cuidado com privacidade. Deve ser realmente única e estável.
Chave artificial (surrogate)	Identificador criado pelo sistema, sem significado de negócio.	id_aluno = 1001, UUID.	Simplifica relacionamentos; ainda pode ser necessário impor UNIQUE a identificadores naturais relevantes.

Exemplo SQL

```
CREATE TABLE aluno (
id_aluno INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
email VARCHAR(255) UNIQUE NOT NULL,
nome VARCHAR(100) NOT NULL
);
```

Nesse exemplo, **id_aluno** é artificial e identifica a linha; **email** tem uma restrição UNIQUE, mas só deve ser usado como chave natural se a regra de negócio garantir que seja estável e apropriado.

Regra prática

Prefira uma chave artificial quando os identificadores do negócio podem mudar ou são compostos. Mantenha restrições UNIQUE para impedir duplicidade nos campos que realmente precisam ser únicos.

05 | Cardinalidade dos relacionamentos

Cardinalidade descreve quantas ocorrências de uma entidade podem estar associadas a ocorrências de outra. Os modelos mais comuns são 1:1, 1:N e N:N.

TIPO	SIGNIFICADO	EXEMPLO
1:1	Um registro de A relaciona-se com no máximo um de B, e vice-versa.	Pessoa e passaporte ativo, conforme as regras do sistema.
1:N	Um registro de A pode relacionar-se com vários de B; cada B pertence a um A (ou a nenhum, se opcional).	Um aluno pode ter vários telefones.
N:N	Vários registros de A podem relacionar-se com vários de B.	Alunos matriculados em várias disciplinas.

Exemplo de relacionamento 1:N

A chave estrangeira fica no lado N em um relacionamento 1:N.

Como interpretar a opcionalidade

- **1** do lado ALUNO: cada telefone deve estar associado a um aluno neste exemplo.
- **N** do lado TELEFONE: um aluno pode ter vários telefones.
- Se o telefone puder existir sem aluno, a regra muda e deve ser refletida no modelo e nas restrições.

06 | Chaves estrangeiras e implementação

Uma chave estrangeira (FK, Foreign Key) conecta registros entre tabelas e protege a integridade referencial. Em um relacionamento 1:N, a FK normalmente fica na tabela do lado N.

Exemplo completo

```
CREATE TABLE aluno (
id_aluno INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
nome VARCHAR(100) NOT NULL
);

CREATE TABLE telefone (
id_telefone INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
id_aluno INT NOT NULL,
numero VARCHAR(25) NOT NULL,
CONSTRAINT fk_telefone_aluno FOREIGN KEY (id_aluno)
REFERENCES aluno(id_aluno)
);
```

A sintaxe de geração automática do identificador varia entre sistemas de banco de dados. Ajuste o exemplo ao SGBD utilizado (por exemplo, PostgreSQL, MySQL ou SQL Server).

Lista de verificação do modelo

- ■ Cada tabela representa um tipo claro de entidade ou fato.
- ■ Cada tabela possui uma chave primária adequada.
- ■ Campos guardam valores atômicos e tipos coerentes.
- ■ Relacionamentos têm cardinalidade e opcionalidade definidas.
- ■ Chaves estrangeiras representam as ligações necessárias.
- ■ Restrições NOT NULL, UNIQUE e CHECK refletem as regras de negócio.
- ■ As formas normais foram avaliadas sem perder de vista o desempenho e os requisitos reais.

Resumo final — SQL fornece os comandos para trabalhar com os dados; a normalização organiza os fatos; PK e FK identificam e conectam registros; e a cardinalidade descreve as regras dos relacionamentos. Esses conceitos trabalham juntos na construção de um banco consistente.